Terminal 3 ADK Testnet Onboarding Report

Author: Mel Okoye (0xJedi)

Date: 8 August 2026 Location: Lagos, Nigeria

Repo: https://github.com/MELcodes99/terminal3-adk-bounty

Tenant DID: did:t3n:9505ad5f……..6ca63eb3

---

0. Summary

Completed the Quickstart and filed seven issues encountered along the way, one of which blocks the documented Quickstart entirely on the current npm latest.

- Quickstart — Complete (after workaround, see BUG-01)
- Set Up Dev Environment — Complete
- Walkthrough steps 1–3 — Complete; step 4 partially blocked (see BUG-08, BUG-09); step 5 not reached
- Issues filed: 7 (1 blocking, 4 major, 2 minor/environmental)
- Beyond first contract — see Section 6

Headline finding: following the Quickstart exactly, on a clean machine, fails. npm install @terminal3/t3n-sdk resolves to 4.30.0, and the documented code throws a TypeError from inside the minified bundle during handshake(). Pinning to 3.11.0 — the newest version the changelog references — fixes it immediately. Any developer following the docs today hits this on their first run.

1. Environment

Field	Value
OS	macOS 12.7.6 (Monterey)
Architecture	x86_64, native (Intel Core i7-4870HQ, 2015 MacBook Pro — not Rosetta)
Node	v24.13.1
npm	11.8.0
@terminal3/t3n-sdk (as installed by docs)	4.30.0
@terminal3/t3n-sdk (working)	3.11.0
Environment target	setEnvironment("testnet")
Shell	zsh
Network	Nigerian ISP; VPN required for browser, VPN off required for npm

Worth recording as a positive: this is a 2015 Intel MacBook running the last macOS version it supports. It compiled the Rust WASM component and registered a TEE contract without any hardware-related friction. The ADK's hardware floor is low, which matters for the developer markets Terminal 3 appears to be recruiting from.

One caveat on BUG-01 below: this is a single machine. The setEnvironment failure is a JavaScript-level configuration problem rather than anything architecture-specific, so it is very likely universal — but this report does not prove that, and it would be worth confirming on Apple Silicon and Linux before treating it as fully general.

2. Quickstart

2.1 Claim API key and credits

Signed in via Google SSO, claimed 20,000 test credits, received API key and DID. Reaching the claim page required a VPN, see BUG-02.

2.2 Project setup

```
mkdir my-t3n-app && cd my-t3n-app
npm init -y
npm pkg set type=module
npm install @terminal3/t3n-sdk tsx
export T3N_API_KEY="<key>"
```

Result: added 148 packages, audited 149 packages in 26s, with 4 vulnerabilities (3 moderate, 1 critical). See BUG-04.

```
added 148 packages, and audited 149 packages in 26s

46 packages are looking for funding
  run `npm fund` for details

4 vulnerabilities (3 moderate, 1 critical)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
macbook@XENON my-t3n-app %
```

Screenshot: fresh install of @terminal3/t3n-sdk@3.11.0

2.3 Connect and authenticate

Used the Quickstart's quickstart.ts verbatim, no modifications.

First run failed. See BUG-01 for the full trace. After pinning the SDK to 3.11.0, the same unmodified code connected successfully.

```
v24.13.1
macbook@XENON my-t3n-app % npm install @terminal3/t3n-sdk@3.11.0

added 1 package, removed 2 packages, changed 1 package, and audited 150 packages in 4s

46 packages are looking for funding
  run `npm fund` for details

4 vulnerabilities (3 moderate, 1 critical)

To address all issues, run:
  npm audit fix

Run `npm audit` for details.
macbook@XENON my-t3n-app % npx tsx quickstart.ts
Connected as: did:t3n:REDACTED
```

Screenshot: pin to 3.11.0 and successful connection (DID redacted)

3. Set Up Development Environment

3.1 Rust + WASM toolchain

```
rustup target add wasm32-wasip2
cargo install wasm-tools
```

Wall-clock time not recorded.

Note: cargo install wasm-tools produces no progress output for roughly two minutes while it compiles its dependency tree. It reads as a hang. Worth a one-line note in the docs.

3.2 TenantClient

TenantClient constructed successfully with baseUrl: getNodeUrl() and the tenantDid read back from the authenticated session.

await tenant.me() — documented on this page and listed as confirmed in the SDK & API Reference — throws TypeError: tenant.me is not a function on 3.11.0. See BUG-05a below.

Runtime introspection of the live object:

```
prototype: [ 'constructor', 'getEnvironment', 'canonicalName',
  'controlPayload', 'executeControl', 'executeBusinessContract' ]
own props: [ 'config', 'tenant', 'maps', 'contracts', 'token' ]
```

me() is absent from the prototype. There is a tenant sub-object, so the method may live one level lower than documented. The client is otherwise functional — maps, contracts, and token are all present and registration proceeded normally — so this reads as a docs/version mismatch rather than a broken client.

Screenshot: TenantClient method list (DID redacted). Also covers the 4.3 registration output below, captured in the same run.

BUG-05a — tenant.me() documented and marked confirmed, absent on TenantClient in 3.11.0

Field	Value
Severity	Major (docs/SDK mismatch)
Where	Set Up Development Environment step 3; SDK & API Reference
SDK version	3.11.0
Already documented?	No

Actual:

```
TypeError: tenant.me is not a function
at <anonymous> (quickstart.ts:37:14)
```

Note on version coverage, this compounds BUG-01 into the more significant finding. 4.30.0 (current npm latest) breaks setEnvironment, and 3.11.0 (the newest version the changelog references) is missing tenant.me(). Taken together, the documentation does not cleanly describe any single shipped version of the SDK. That is the underlying issue; BUG-01 and BUG-05a are two symptoms of it.

The SDK contradicts itself, not just the docs. me() is declared in the shipped type definitions — node_modules/@terminal3/t3n-sdk/dist/index.d.ts line 3303 reads me(): Promise<unknown>; on the TenantClient class — but is absent from the runtime prototype, as the introspection in Section 3.2 shows. TypeScript therefore compiles a call to tenant.me() without complaint, and it fails only at runtime. A type

definition promising a method the bundle does not ship is worse than a stale doc page: the compiler actively confirms the wrong thing.

Suggested fix: State a supported SDK version range at the top of the Get Started section, verify the reference page's "confirmed" claims against a specific published version, and check that `index.d.ts` is generated from the same build as `index.esm.js` — the mismatch here suggests it is not.

4. Walkthrough

4.1 Write your TEE contract

Cloned the reference implementation as the docs direct, into a sibling folder of my-t3n-app:

```
git clone https://github.com/Terminal-3/z-tenant-flight.git
```

Structure matched the documented layout, with one addition — see BUG-07.

Host interfaces declared in wit/world.wit: tenant-context@1.0.0, logging@2.1.0, kv-store@2.1.0, http@2.1.0, http-with-placeholders@2.1.0.

Screenshot: ls -R src wit — shows the third WIT dependency from BUG-07 too

4.2 Build your TEE contract

```
cargo build --release --target wasm32-wasip2
```

Built clean on the first attempt, no warnings blocking. Artifact:

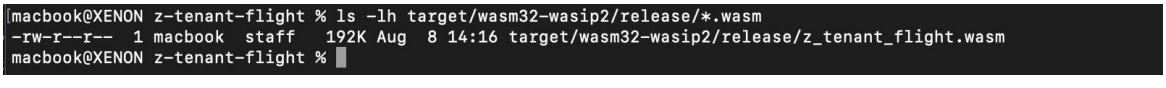
```
192K target/wasm32-wasip2/release/z_tenant_flight.wasm
```

Verified as a genuine WASM component rather than a bare module:

```
wasm-tools validate target/wasm32-wasip2/release/z_tenant_flight.wasm
wasm-tools component wit target/wasm32-wasip2/release/z_tenant_flight.wasm
```

The component's declared world exports z:tenant-flight/contracts@0.4.0. Its import list surfaced BUG-06.

Build succeeded on first attempt; timing and toolchain versions not recorded.



Screenshot: built WASM artifact, 192K z_tenant_flight.wasm

4.3 Register your TEE contract

Registered from quickstart.ts via tenant.contracts.register(), using the sibling-folder WASM path exactly as documented.

```
tail:    jedi-flight
version: 0.1.1
result:  registered z:<tid>:jedi-flight as contract id 556
```

Succeeded on the first attempt, no workaround needed. Short tail chosen deliberately per the docs' note about undocumented downstream length limits. (Version shown as 0.1.1: the contract was re-registered at 0.1.1 when capturing this screenshot, since 0.1.0 was already registered under this tail from an earlier run — hence contract id 556 rather than the id from that first registration.)

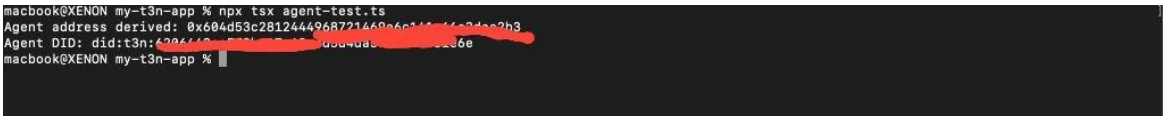
Worth flagging as a platform gap rather than a bug: the docs state there is no API to fetch a tail's current contract_id after re-registration, and that map ACLs scoped to an old contract_id can be left pointing at a stale ID. That places the burden on every tenant to keep their own external record of every contract_id they have ever registered. A read endpoint would remove a whole class of silent ACL failures.

Registration screenshot: see the combined capture in Section 3.2 above, taken in the same script run.

4.4 Invoke your TEE contract

Partially completed. Agent identity established; full end-to-end invocation blocked by prerequisites the walkthrough does not provide — see BUG-08 and BUG-09.

Agent session — working. Authenticated a separate agent session and received an auto-provisioned agent DID distinct from the tenant DID.



Screenshot: agent session and auto-provisioned agent DID (redacted)

Not completed: the agent-auth-update grant and the search-offers / book-offer calls, both of which require a Duffel API key seeded into a secrets KV map that the walkthrough never instructs the reader to obtain.

4.5 Test your TEE contract

Not reached — blocked by the same prerequisites as 4.4.

5. Issues filed

Severity key: Blocking — cannot proceed on the documented path. Major — documented behaviour is wrong or the error is misleading. Minor — cosmetic. Environmental — outside Terminal 3's code but affects onboarding.

BUG-01 — Documented Quickstart fails on current npm latest (4.30.0)

Field	Value
Severity	Blocking
Where	Quickstart, step 3
SDK version failing	4.30.0 (resolved by npm install @terminal3/t3n-sdk with no version specifier)
SDK version working	3.11.0
Node	v24.13.1
request_id	None — the failure is client-side, before any request is sent
Already documented?	No. Absent from Common Errors and from the Changelog, as of 8 Aug 2026

Steps to reproduce

- Follow the Quickstart project setup exactly, including npm install @terminal3/t3n-sdk tsx with no version pin.
- Create quickstart.ts with the Quickstart's code block, copied verbatim and unmodified.
- Run npx tsx quickstart.ts.

Expected:

```
Connected as: did:t3n:…
```

Actual:

```
TypeError: Cannot read properties of undefined (reading 'unsafe_trust_server')
  at isUnsafeTrustServer (node_modules/@terminal3/t3n-sdk/dist/index.esm.js:2:1013665)
  at assertNodeTrusted (node_modules/@terminal3/t3n-sdk/dist/index.esm.js:2:1111694)
  at <anonymous> (node_modules/@terminal3/t3n-sdk/dist/index.esm.js:2:1112626)
  at process.processTicksAndRejections (node:internal/process/task_queues:103:5)
  at async T3nClient.handleGuestToHost (…:2:1158345)
  at async T3nClient.handleWasmRequest (…:2:1156711)
  at async T3nClient.runFlow (…:2:1155668)
  at async T3nClient.handshake (…:2:1131952)
  at async <anonymous> (quickstart.ts:23:1)
```

Analysis

assertNodeTrusted calls isUnsafeTrustServer, which dereferences a property on an object that is undefined at that point. The failure is inside handshake(), before any network request, which rules out connectivity and credentials. The most likely reading is that setEnvironment("testnet") no longer populates the config object that assertNodeTrusted reads in 4.x — a breaking change between 3.x and 4.x that the Quickstart was never updated for.

Supporting evidence: the Changelog references only 3.5.2, 3.9.0, and 3.11.0, and explicitly states no confirmed release history exists. The docs appear to have been written against 3.x while npm latest is a major version ahead.

Workaround

```
npm install @terminal3/t3n-sdk@3.11.0
```

The same unmodified Quickstart code then connects successfully on the first attempt.

Suggested fix: Either update the Quickstart for the 4.x API surface, or pin the version in the install command (npm install @terminal3/t3n-sdk@3.11.0 tsx) until it is updated. Separately, an invalid or missing environment config should raise a readable error naming the problem, rather than a TypeError from a minified bundle — the current failure gives a first-time developer nothing actionable to search for.

Impact: Every developer following the Quickstart on a clean machine hits this. For an onboarding funnel, it is the first thing they experience.

Screenshot: TypeError trace on SDK 4.30.0

BUG-02 — Claim page does not render from a Nigerian ISP; resolves via VPN

Field	Value
Severity	Major (regional onboarding blocker)
Where	go.terminal3.io/adk-community and terminal3.io/products/agent-developer-kit
Already documented?	No

Behaviour

Page returned HTML but did not render, a blank or partially-drawn page, consistent with client-side JS or `_next` chunks failing to load. Reproduced on both the shortlink and the direct URL, in Chrome, across three attempts. Connecting through a VPN with a US exit resolved it immediately, on the same machine and browser. No DevTools capture was taken at the time, so the specific failing request is not identified here. Worth confirming across other Nigerian ISPs and other African markets before assuming it is isolated to one connection.

BUG-03 — npm audit fails with a TLS error while VPN is connected

Field	Value
Severity	Environmental
Already documented?	No

Actual:

```
npm warn audit request to https://registry.npmjs.org/-/npm/v1/security/advisories/bulk failed,
reason: 00C6DC1401000000:error:0A0003FC:SSL routines:ssl3_read_bytes:
ssl/tls alert bad record mac
npm error audit endpoint returned an error
```

Succeeded with the VPN disconnected.

Note

Not a Terminal 3 defect. Recorded because it combines with BUG-02 into a genuine regional friction pattern: the browser flow needs a VPN, the npm flow breaks with one. A developer in this region has to know to toggle it between steps, and nothing in the docs would lead them there. Worth a line in a troubleshooting section.

```
Error: Failed to fetch node status from https://cn-api.sg.testnet.t3n.terminal3.io/status: fetch failed
    at fetchStatus (/Users/macbook/my-t3n-app/node_modules/@terminal3/t3n-sdk/dist/index.esm.js:2:960083)
    at process.processTicksAndRejections (node:internal/process/task_queues:103:5)
    at async fetchMlKemPublicKey (/Users/macbook/my-t3n-app/node_modules/@terminal3/t3n-sdk/dist/index.esm.js:2:96
0544)
    at async <anonymous> (/Users/macbook/my-t3n-app/node_modules/@terminal3/t3n-sdk/dist/index.esm.js:2:966850)
    at async T3nClient.handleGuestToHost (/Users/macbook/my-t3n-app/node_modules/@terminal3/t3n-sdk/dist/index.esm
.js:2:993194)
    at async T3nClient.handleWasmRequest (/Users/macbook/my-t3n-app/node_modules/@terminal3/t3n-sdk/dist/index.esm
.js:2:991561)
    at async T3nClient.runFlow (/Users/macbook/my-t3n-app/node_modules/@terminal3/t3n-sdk/dist/index.esm.js:2:9905
16)
    at async T3nClient.handshake (/Users/macbook/my-t3n-app/node_modules/@terminal3/t3n-sdk/dist/index.esm.js:2:97
7293)
    at async <anonymous> (/Users/macbook/my-t3n-app/quickstart.ts:23:1)
```

Screenshot: fetch failed / node status error

BUG-04 — Fresh SDK install surfaces 1 critical and 3 moderate advisories

Field	Value
Severity	Minor (supply-chain hygiene)
Already documented?	No

Dependency chain

```
@terminal3/t3n-sdk
  └── @bytecodealliance/jco
        └── @bytecodealliance/componentize-js
              └── @bytecodealliance/weval
                    └── decompress  ← critical, no fix available
```

decompress carries two unpatched advisories: GHSA-mp2f-45pm-3cg9 and GHSA-h39j-r5qq-r9mm, both Zip Slip / arbitrary file write via archive extraction.

Assessment

Transitive, and part of the Bytecode Alliance WASM toolchain rather than Terminal 3's own code. Build-time only — it does not sit on a path that handles untrusted user input at runtime, so the practical exploitability here is low. Filing it as hygiene, not as an exploitable vulnerability in the SDK.

The reason it is worth raising anyway: every developer onboarding sees 1 critical on their very first install. For a company whose product is privacy and security infrastructure, that is a bad first impression even when the finding is benign. Options are pinning around the affected jco range, or a one-line note in the docs pre-empting it.

```
# npm audit report

decompress  *
Severity: critical
Decompress: Archive extraction can create files and links outside of the target directory - https://github.com/advisories/GHSA-mp2f-45pm-3cg9
decompress: Arbitrary File Write via Archive Extraction (Zip Slip) - https://github.com/advisories/GHSA-h39j-r5qq-r9mm
fix available via `npm audit fix`
node_modules/decompress
  @bytecodealliance/weval  *
  Depends on vulnerable versions of decompress
  node_modules/@bytecodealliance/weval
    @bytecodealliance/componentize-js  0.13.0 - 0.18.5 || >=0.19.4-rc.1
    Depends on vulnerable versions of @bytecodealliance/weval
    node_modules/@bytecodealliance/componentize-js
      @bytecodealliance/jco  1.7.0 - 1.15.0 || >=1.18.0
      Depends on vulnerable versions of @bytecodealliance/componentize-js
      node_modules/@bytecodealliance/jco

4 vulnerabilities (3 moderate, 1 critical)

To address all issues, run:
  npm audit fix
```

Screenshot: npm audit output showing the decompress advisory chain

BUG-05 — Docs instruct developers to remove the hex::encode their own reference implementation requires

Field	Value
Severity	Major — leads to a wrong KV namespace path
Where	Write your TEE contract, "Reading secrets from the secrets KV map" and "Key Design Rules"
Already documented?	No

The contradiction

The docs state that tenant_did() already returns the tid as a string, instruct the reader not to hex::encode it, and characterise doing so as "a real bug some teams have hit" that "silently produces a map path that doesn't match anything you created."

Three independent sources contradict this.

1. The WIT ABI, read directly out of the compiled component:

```
package host:tenant@1.0.0 {
  interface tenant-context {
    tenant-did: func() -> list<u8>;
  }
}
```

list<u8> is bytes, not a string.

2. z-tenant-flight/src/search.rs:

```
let tid = tenant_context::tenant_did();
let map_name = alloc::format!("z:{}:secrets", hex::encode(&tid));
```

3. z-tenant-flight/src/booking.rs — identical, same two lines.

The official reference implementation hex-encodes in both places. The documented code sample does not.

Impact

A developer following the documented sample, with tid as Vec<u8>, fails to compile — Vec<u8> has no Display impl, so format!("z:{}:secrets", tid) is rejected. If they instead reach for String::from_utf8, they build a map path that does not correspond to any namespace they created.

The more likely real-world failure is the inverse of the one documented: because the docs frame hex::encode as a known bug, a developer who copied the working reference implementation may remove correct code on the docs' advice and break a previously working contract. The warning describes the correct behaviour as the bug.

For a platform where this value selects which tenant KV namespace is read, this is not cosmetic.

Suggested fix: Correct the sample and the Key Design Rules bullet to match the ABI and the reference implementation, and remove the warning against hex::encode. If tenant_did() is intended to return a string in a future host version, state the version boundary explicitly.

```
package root:component;

world root {
  import host:tenant/tenant-context@1.0.0;
  import host:interfaces/logging@2.1.0;
  import host:interfaces/kv-store@2.1.0;
  import host:interfaces/http@2.1.0;
  import host:interfaces/http-with-placeholders@2.1.0;
  import wasi:io/poll@0.2.6;
  import wasi:io/error@0.2.6;
  import wasi:io/streams@0.2.6;
  import wasi:cli/environment@0.2.6;
  import wasi:cli/exit@0.2.6;
  import wasi:cli/stdin@0.2.6;
  import wasi:cli/stdout@0.2.6;
  import wasi:cli/stderr@0.2.6;
  import wasi:cli/terminal-input@0.2.6;
  import wasi:cli/terminal-output@0.2.6;
  import wasi:cli/terminal-stdin@0.2.6;
  import wasi:cli/terminal-stdout@0.2.6;
  import wasi:cli/terminal-stderr@0.2.6;

  export z:tenant-flight/contracts@0.4.0;
}
package host:tenant@1.0.0 {
  interface tenant-context {
    tenant-did: func() -> list<u8>;
  }
}
```

Screenshot: wasm-tools component wit output showing the list<u8> signature

```
grep -A 8 "tenant_did()" src/booking.rs
    let tid = tenant_context::tenant_did();
    let map_name = alloc::format!("z:{}:secrets", hex::encode(&tid));
    let bytes = kv_store::get(&map_name, b"duffel_api_key")
        .map_err(|e| alloc::format!("kv read: {e}"))?
        .ok_or("duffel_api_key not found in z:<tid>:secrets — populate it via the tenant SDK before use")?;
    alloc::string::String::from_utf8(bytes).map_err(|e| e.to_string())
}

#[cfg(target_arch = "wasm32")]
    let tid = tenant_context::tenant_did();
    let map_name = alloc::format!("z:{}:secrets", hex::encode(&tid));
    let bytes = kv_store::get(&map_name, b"duffel_api_key")
        .map_err(|e| alloc::format!("kv read: {e}"))?
        .ok_or("duffel_api_key not found in z:<tid>:secrets — populate it via the tenant SDK before use")?;
    alloc::string::String::from_utf8(bytes).map_err(|e| e.to_string())
}

#[cfg(target_arch = "wasm32")]
```

Screenshot: grep -A 8 tenant_did() output, search.rs and booking.rs, both hex-encoding the tid

BUG-06 — Compiled component imports 14 WASI interfaces not declared in world.wit

Field	Value
Severity	Major — capability model looser than documented, or reference contract will not load
Where	Capabilities come from your WIT imports; reference repo z-tenant-flight
Already documented?	No

Declared in wit/world.wit: tenant-context, logging, kv-store, http, http-with-placeholders — five interfaces.

Actually imported by the compiled component, per wasm-tools component wit: those five, plus fourteen more —

```
wasi:io/poll@0.2.6, wasi:io/error@0.2.6, wasi:io/streams@0.2.6,
wasi:cli/environment@0.2.6, wasi:cli/exit@0.2.6,
wasi:cli/stdin@0.2.6, wasi:cli/stdout@0.2.6, wasi:cli/stderr@0.2.6,
wasi:cli/terminal-input@0.2.6, wasi:cli/terminal-output@0.2.6,
wasi:cli/terminal-stdin@0.2.6, wasi:cli/terminal-stdout@0.2.6,
wasi:cli/terminal-stderr@0.2.6
```

Why this matters

The docs state that imported interfaces are the contract's entire capability set, that there is no separate manifest, and that the host "refuses to load a contract that imports an interface its tenant world does not provide."

Both readings are a problem:

- If the host enforces that rule strictly, the official reference implementation should fail to load, since it imports thirteen interfaces its declared world does not list.
- If the host tolerates undeclared WASI imports, then the capability model is broader than documented. wasi:cli/environment in particular is host environment access appearing in a contract that never asked for it — on a platform whose security claim is that contracts see only what they declare, that gap deserves an explicit statement rather than silence.

Most likely these are injected by the wasm32-wasip2 target and stripped or ignored at load time. That is a reasonable design. It is simply not written down anywhere, and a developer auditing their own contract's capability surface with wasm-tools will see this and have no way to tell whether it is expected.

Suggested fix: State explicitly how the host treats wasi:* imports that the declared world does not list — stripped, ignored, or denied — and note that wasm32-wasip2 adds them automatically.

```
package root:component;

world root {
  import host:tenant/tenant-context@1.0.0;
  import host:interfaces/logging@2.1.0;
  import host:interfaces/kv-store@2.1.0;
  import host:interfaces/http@2.1.0;
  import host:interfaces/http-with-placeholders@2.1.0;
  import wasi:io/poll@0.2.6;
  import wasi:io/error@0.2.6;
  import wasi:io/streams@0.2.6;
  import wasi:cli/environment@0.2.6;
  import wasi:cli/exit@0.2.6;
  import wasi:cli/stdin@0.2.6;
  import wasi:cli/stdout@0.2.6;
  import wasi:cli/stderr@0.2.6;
  import wasi:cli/terminal-input@0.2.6;
  import wasi:cli/terminal-output@0.2.6;
  import wasi:cli/terminal-stdin@0.2.6;
  import wasi:cli/terminal-stdout@0.2.6;
  import wasi:cli/terminal-stderr@0.2.6;

  export z:tenant-flight/contracts@0.4.0;
}
package host:tenant@1.0.0 {
  interface tenant-context {
    tenant-did: func() -> list<u8>;
  }
}
```

Screenshot: full wasm-tools component wit output (same capture as BUG-05)

BUG-07 — Reference repo ships a third WIT dependency the docs' structure diagram omits

Field	Value
Severity	Minor (doc drift)
Where	Write your TEE contract, "Repository Structure"
Already documented?	No

The docs describe wit/deps/ as containing host-interfaces-2.1.0/ and host-tenant-1.0.0/. The cloned repo also contains host-outbox-1.0.0/.

Compounding it slightly: the SDK & API Reference lists outbox as "coming soon," while the vendored package is present in the reference project today. Not a defect, but a developer checking which capabilities are available gets two different answers depending on which page they read.

Documentation observations (not bugs)

- cargo install wasm-tools produces no output for around two minutes. Reads as a hang.
- No supported Node version is stated anywhere in the docs. Given BUG-01 involves a WASM toolchain that historically lags new Node releases, a stated minimum and maximum would help.
- The Quickstart's code block needs its "this is TypeScript, not shell commands" warning to be harder to miss — it currently sits after the paste target.

6. Beyond the first contract — use case proposal

6.1 Second contract

Name: rx-refill-agent

Exercises: kv-store read (member ID/DOB stored in a secrets map) plus an outbound http-with-placeholders call to a pharmacy/PBM refill API

Repo path: contracts/rx-refill-agent/

[SCREENSHOT: registered and invoked — still needed]

6.2 Proposed use case

Primitive it builds on: http-with-placeholders. {{member.*}} markers get resolved host-side inside the enclave, so plaintext PII never enters WASM memory.

Problem

An AI agent handling something like “refill my prescription” needs to pass a member ID, date of birth, and medication details to a pharmacy or PBM API. If that PII ever sits in the agent's LLM context or a normal backend's memory or logs, it's exposed to prompt injection, provider-side logging, and real regulatory liability around health data.

Flow on T3N

- The agent calls the contract with a refill request containing only placeholder tokens, like member.id and member.dob, never the raw values.
- The contract issues the pharmacy API call through http-with-placeholders. The host resolves those placeholders into real values only inside the enclave, at request time.
- The response, refill status and pickup window, comes back to the agent with sensitive fields filtered out contract side before it ever leaves the TEE boundary.

Why this needs a TEE rather than a normal backend

A conventional backend still needs some service to hold plaintext PII in memory, and usually to log or trace it somewhere. The layer around an AI agent, which often calls out to a third party LLM API, should never see that data at all. A TEE contract lets placeholder resolution happen in an execution environment that can be independently attested. The pharmacy partner can verify exactly what code touched the data, instead of just trusting the AI vendor's word for it. That's a trust minimization problem, not just an encryption problem, and only an attestable enclave actually solves it.

What's missing from the ADK to build this today

There's no documented pattern for a third party, like a PBM, to pre provision per user secrets into a contract's secrets map before the agent ever runs. The walkthrough only covers secrets seeded manually by the developer. http-with-placeholders resolves placeholders on the way out, but there's no equivalent for redacting sensitive fields in the response if the upstream API returns more than the contract should expose. BUG-08 and BUG-09 already ran into this same gap with Duffel. The real blocker isn't this specific use case, it's that there's no documented way to get a real API key or credential into a contract's secrets ahead of time.

7. Repo contents

Path	What it is
quickstart.ts	Quickstart + TenantClient, single file per the docs
contracts/jedi-flight/	First TEE contract
contracts/rx-refill-agent/	Second contract (bonus)
screenshots/	Numbered to match this document
BUGS.md	This bug section, in-repo
README.md	Setup and how to run

Pre-submit checklist

☐ API key appears nowhere in the repo, this doc, or any screenshot — check every screenshot twice

☐ Gmail address cropped from the claim-page screenshot

☐ .env gitignored and git history clean of any key

☐ Every screenshot legible and matched to its section

☐ Repo public — verified in a private window

☐ Doc set to anyone-with-link-can-view — verified in a private window

☐ Doc links to repo, README links back to doc

☐ BUG-01 also reported in the developer Telegram — a blocking quickstart bug is worth telling them about before judging